

Analyse numérique et calcul scientifique - 1

TP 3 (3h) Équations différentielles ordinaires

Objectifs

- Programmer de façon autonome en Python.
- Se familiariser avec les méthodes de résolution des équations différentielles ordinaires (EDO): Euler, Runge-Kutta et multipas.
- Etudier la stabilité et l'ordre de convergence des schémas numériques de résolution d'EDOs.
- Appliquer à des problèmes issus de la mécanique, en particulier pour des EDOs de deuxième ordre.

Preparation Compte-Rendu

En complément des consignes des TP 1 et 2, vous devez indiquer explicitement dans l'introduction dans quelle mesure vous avez utilisé des outils d'IA générative (par exemple : ChatGPT, Gemini, etc.) dans ce travail : génération de code, relecture grammaticale, explications théoriques, etc. Vous devez **toujours** adapter et évaluer de manière critique les résultats fournis par ces outils. Toute absence de déclaration précise, si elle est constatée ultérieurement, sera **sanctionnée**.

La date limite pour les rapports des TP 3 et 4 est fixée au **28/12 à 23h59**. Dans la journée du 29/12, je vous écrirai pour confirmer la bonne réception de vos envois et pour vous communiquer les informations utiles concernant la soutenance du **06/01**, notamment le sous-ensemble de questions du TP sur lequel vous concentrer pour votre préparation.

1 Stabilité absolue

On considère le problème modèle (problème de Dahlquist):

$$\begin{cases} y'(t) = \lambda y(t) \\ y(0) = 1 \end{cases} \quad (1)$$

pour lequel on a $f(t, y(t)) = \lambda y(t)$ et dont la solution exacte est $y(t) = \exp(\lambda t)$ pour $\lambda \in \mathbb{R}^-$ (le cas général complexe étant négligé pour l'instant). Pour les implémentations numériques, vous êtes libres de choisir le λ que vous convient, qui doit être fixé au début, ainsi comme la plage maximale de t ($t_{max} > 0$), tel que $t \in [0, t_{max}]$. Un plan d'étude est proposé ci-dessous, vous devez le suivre tout en commentant les résultats obtenus.

1. Pour la méthode d'Euler explicite, tracer les résultats de ces simulations pour trois choix de Δt (pour le λ donné): i) stable sans oscillations, ii) stable mais avec des oscillations, et iii) instable. Veillez à adapter la valeurs de λ et t_{max} , de façon à facilement visualiser le comportements souhaités (astuce : au lieu de définir Δt , le plus simple est définir le nombre de divisions de l'intervalle $[0, t_{max}]$ tel que les Δt soient dans les régions choisis).
2. Pour les mêmes valeurs de Δt de 1.1), appliquer la méthode d'Euler implicite. Comme l'EDO est simple, la programmation d'Euler implicite est sans difficultés. Tracer les résultats pour constater sa propriété théorique vis-à-vis à sa stabilité.

3. Pour les mêmes valeurs de Δt de 1.1), appliquer la méthode Crank-Nicholson (Adams-Moulton d'ordre 2). Comme l'EDO est simple, la programmation est sans difficultés. Tracer les résultats pour constater sa propriété théorique vis-à-vis a sa stabilité.
4. Le script fourni permet de visualiser les régions de stabilité pour les méthodes d'Euler explicite, et Runge-Kutta 2 et 4 dans le cadre $\lambda \in \mathbb{C}$: i) quel d'entre les trois méthodes a la région stable la plus grande?; ii) à quoi représente la variable z en fonction de λ et Δt ? iii) justifiez pour le cas RK2 le polynôme complexe implémenté.

2 Pendule

En appliquant le PFD au pendule de longueur ℓ et accélération de la pesanteur g , on a l'EDO de deuxième ordre

$$\ddot{\theta}(t) + \frac{g}{\ell} \sin \theta(t) = 0, \quad \theta(t) = \theta_0, \dot{\theta}(t) = \omega_0, \quad (2)$$

1. Appliquer la méthode d'Euler explicite, RK2 (Heun) et RK4 pour ce problème (en format vectoriel) pour un jeu de paramètres choisi (indiquez le). Vous devez simuler pendant environ une période (astuce : vous pouvez estimer la période en utilisant la théorie des cas de petites angles : $T = \frac{2\pi}{\omega}$ avec $\omega = \sqrt{\frac{g}{\ell}}$).
2. Vérifier que l'énergie mécanique se conserve (rappel $K = \frac{1}{2}m(\ell\dot{\theta})^2, U = mgl(1 - \cos \theta), E_m = K + U$).
3. Utilisez la méthode du saute-mouton (code fourni) et vérifiez la conservation d'énergie.
4. Implémenter la méthode de Newmark pour la résolution de l'EDO linéarisée pour petits angles, incluant un amortissement b , donnée par, tel que $\ddot{\theta}(t) + b\dot{\theta}(t) + \frac{g}{\ell}\theta(t) = 0$, pour $\beta = 1/4, \gamma = 1/2$. Commenter les résultats obtenus en simulant le système sur plusieurs périodes, pour une valeur de b assurant une solution oscillatoire mais amortie.

3 Convergence méthodes multipas (Adams-Bashford)

Pour étudier numériquement l'ordre de convergence d'une méthode, il est nécessaire de disposer de sa solution exacte (analytique). Cependant, sauf dans des cas très spécifiques, déterminer la solution d'une EDO est souvent laborieux, voire impossible. Pour contourner cette difficulté, nous allons créer un cas test en utilisant ce que l'on appelle la *méthode des solutions construites*. Prenons par exemple $y(t) = \exp t^2 \cos(t^3)$ et cherchons $y'(t)$

$$\begin{aligned} y'(t) &= 2t \exp t^2 \cos t^3 - 3t^2 \exp t^2 \sin t^3 \\ &= 2ty(t) - 3t^2 y(t) \tan t^3 := f(t, y(t)) \end{aligned}$$

Il faut juste appliquer des méthodes étudiés pour $f(t, y(t))$ trouvé en sachant $y(0) = 1$. On va donc s'intéresser au erreur dite $E_{L^2}^1$

$$E_{L^2} = \sqrt{\int_0^{t_{max}} |\tilde{e}(t)|^2 dt} \approx \sqrt{\frac{\Delta t}{2} \sum_{n=0}^{N-1} (e_{n+1}^2 + e_n^2)}, \quad \text{où } e_n = |u_n - y(t_n)| \quad \forall n = 0, \dots, N. \quad (3)$$

1. Implémenter une fonction générique pour la méthode Adams-Bashford pour 1 à 4 points. Parmi d'autres arguments, la fonction devra recevoir le nombre p des points et la fonction $f(t, y)$ à deux variables. Il faudra noter que les méthodes multipas nécessitent initialisation de plus qu'un valeur précédent. Vous pouvez utiliser la méthode RK4 pour calculer ces points manquantes.
2. Postuler une solution exacte $y(t)$ et trouver la fonction $f(t, y)$ correspondant (une autre). Choisir $y(t)$ suffisamment compliqué pour rendre le problème intéressant. C'est important de d'écrire la dépendance en $y(t)$ explicitement sur f , de façon qu'on puisse tester la propagation d'erreurs dans chaque méthode.

¹ E_{L^2} est l'application de la méthode du trapèze pour le calcul de la norme euclidienne au sens du produit interne des de fonctions.

3. Tracer les courbes erreur vs Δt en échelle logarithmique dans deux axes.
4. Redériver la méthode de Adams-Bashford 2 points.

Annexes

A Méthodes à un pas

Euler

$$y_{n+1} = y_n + hf(t_n, y_n) \quad (\text{explicite}) \quad (4)$$

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}) (\text{implicite}) \quad (5)$$

Runge-Kutta

$$y_{n+1} = y_n + h\Phi(x_n, y_n; h) \quad \Phi(x_n, y_n; h) = \sum_{r=1}^m c_r k_r. \quad (6)$$

avec

$$k_1 = f(t_n, y_n), \quad \dots \quad k_r = f\left(t_n + h a_r, y_n + h \sum_{s=1}^{r-1} b_{r,s} k_s\right).$$

0				
a_2	$b_{2,1}$			
a_3	$b_{3,1}$	$b_{3,2}$		
$\vdots$	$\vdots$	$\vdots$	$\ddots$	
a_m	$b_{m,1}$	$b_{m,2}$	$\dots$	$b_{m,m-1}$
	c_1	c_2	$\dots$	$c_{m-1} \quad c_m$

RK2 (Heun)		
0		
1/2	1/2	
	0	1

RK2 (Euler Modifié)		
0		
1	1	
	1/2	1/2

RK3			
0			
1/3	1/3		
2/3	0	2/3	
	1/4	0	3/4

RK4				
0				
1/2	1/2			
1/2	0	1/2		
1	0	0	1	
	1/6	1/3	1/3	1/6

B Méthodes multipas

Les méthodes sont déterminées par $r \geq 0$ (entier), tel que l'ordre de convergence est donnée $\mathcal{O}(h^{r+1})$. Avec la notation $f_n = f(x_n, y_n)$, on pose

$$(\text{Adams-Bashfort}) \quad y_{n+1} = y_n + h \sum_{j=0}^r \bar{b}_{j,r} f_{i-j}, \quad (\text{Adams-Moulton}) \quad y_{n+1} = y_n + h \sum_{j=0}^r \bar{b}_{j,r} f_{i-j+1} \quad (7)$$

Adams-Bashford				
r	$\bar{b}_{0,r}$	$\bar{b}_{1,r}$	$\bar{b}_{2,r}$	$\bar{b}_{3,r}$
0	1			
1	3/2	-1/2		
2	23/12	-4/3	5/12	
3	55/24	-59/24	37/24	-3/8

Adams-Moulton				
r	$\bar{b}_{0,r}$	$\bar{b}_{1,r}$	$\bar{b}_{2,r}$	$\bar{b}_{3,r}$
0	1			
1	1/2	1/2		
2	5/12	8/3	-1/12	
3	9/24	19/24	-5/24	1/24

C Extraits codes

```
# t doit être un array (e.g. par np.linspace)
# y0 est vectorielle pour généralité
def euler_exp(f, y0, t):
    n = len(t)
    y = np.zeros((n, len(y0)))
    y[0] = y0
    for i in range(n - 1):
        y[i+1] = y[i] + (t[i+1] - t[i]) * f(y[i], t[i])
    return y
```

```
# t doit être un array (e.g. par np.linspace)
# y0 est vectorielle pour généralité
def rungekutta4(f, y0, t):
    n = len(t)
    y = np.zeros((n, len(y0)))
    y[0] = y0
    for i in range(n - 1):
        h = t[i+1] - t[i]
        k1 = f(y[i], t[i])
        k2 = f(y[i] + k1 * h / 2., t[i] + h / 2.)
        k3 = f(y[i] + k2 * h / 2., t[i] + h / 2.)
        k4 = f(y[i] + k3 * h, t[i] + h)
        y[i+1] = y[i] + (h / 6.) * (k1 + 2*k2 + 2*k3 + k4)
    return y
```

```
import numpy as np
import matplotlib.pyplot as plt

# -- Fonction de stabilité Euler explicite --
def R_euler(z):
    return 1 + z

# -- Fonction de stabilité RK4 --
def R_rk4(z):
    return 1 + z + z**2/2 + z**3/6 + z**4/24

# -- Fonction de stabilité RK2 --
def R_rk2(z):
    return 1 + z + z**2/2

# -- Domaine du plan complexe --
x = np.linspace(-3, 3, 800)
y = np.linspace(-3, 3, 800)
X, Y = np.meshgrid(x, y)
Z = X + 1j * Y

# -- Normes --
euler_level = np.abs(R_euler(Z))
rk2_level = np.abs(R_rk2(Z))
rk4_level = np.abs(R_rk4(Z))

# -- Figures --
plt.figure(1, (5, 5))
plt.contour(X, Y, euler_level, levels=[1],
            colors='blue')
plt.contour(X, Y, rk4_level, levels=[1], colors
            ='red')
plt.contour(X, Y, rk2_level, levels=[1], colors
            ='green')
plt.xlabel("Re(z)")
plt.ylabel("Im(z)")
plt.grid()
```

```
def verlet(f, y0, t): # ou saute-mouton
    n = len(t)
    y = np.zeros((n, len(y0)))
    y[0,:] = y0 # correspond to time t0

    for i in range(n-1):
        h = t[i+1] - t[i]
        aold = f(y[i], t[i])[1]
        y[i+1,0] = y[i,0] + h*y[i,1] + 0.5*h**2*aold
        # f_1 is the acceleration
        anew = f(y[i+1], t[i+1])[1]
        y[i+1,1] = y[i,1] + 0.5*h*(anew + aold)

    return y
```

```
# ...

methods = { 'EE': euler_exp,
            'RK4': rungekutta4}

# ...

y_exact = lambda t: np.exp(t**2)*np.cos(t**3)
f = lambda y, t: 2*t*y - 3*t**2*y*np.tan(t
**3)

# np.trapz is the numpy's trapezoidal rule
for integrals
L2norm = lambda x,t: np.sqrt(np.trapz(x**2,t)
)

# if N increases with the power of 2, h
decreases by half in each increment.
Nlist = np.array([2**i for i in range(1,10)])
hlist = t_max/Nlist

Iapp = {}
error = {key: [] for key in methods.keys()}
for N in Nlist:
    t = np.linspace(0., t_max, N)
    yex = y_exact(t)
    for m in methods.keys():
        ode_fun = methods[m]
        y_num = ode_fun(f, [y0], t)[: ,0]
        error[m].append(L2norm(y_num-yex, t))

# order of convergence evaluated between the
last two errors
for m in methods.keys():
    print("order {0} : {1}".format(m, np.log2(
        error[m][-2]/error[m][-1])))

plt.figure(2)
for m in methods.keys():
    plt.plot(hlist, error[m], '-o', label = m)
plt.yscale('log')
plt.xscale('log')
plt.xlabel('$h$')
plt.xlabel('$L2$ error$')
plt.legend()
plt.grid()
```